[데이터통신 7주차 과제]

[UDP/TCP Socket Programming]

-학번:2371214

-이름:이지형

-트랙: 빅데이터&모바일소프트웨어

-분반: 데이터통신B반

<과제내용>

**2장 7절 내용을 참조하여, 'echo' client-server 프로그램을 구현하고,
실행화면과 소스코드를 제출.**

- TCP, UDP 로 각각 소켓을 구현함. 따라서 source code 2쌍 - 4개 구현.

1-1. UDP_server.py 소스코드 제출

```
from socket import *

student_id = "2371214"

student_name = "이 지 형 "


 # 서버 포트 설정

serverPort = 12000

# UDP 소켓 생성 및 포트 바인딩

serverSocket = socket(AF_INET, SOCK_DGRAM)

serverSocket.bind(('', serverPort))
print(f"=========================================")

print(f" UDP Server Started by [{student_id} {student_name}]")

print(f" The server is ready to receive (Port: {serverPort})")
print(f"=========================================₩n")

while True:

        # 데이터 및 클라이언트 주소 수신

        message, clientAddress = serverSocket.recvfrom(2048)

        # 수신한 데이터 디코딩 후 출력

        decoded_message = message.decode()

        print(f"[Received from {clientAddress}]: {decoded_message}")
        modifiedMessage = decoded_message.upper()
        serverSocket.sendto(modifiedMessage.encode(), clientAddress)
```

1-2.UDP_client.py 소스코드 제출

```
from socket import *

student_id = "2371214"
student_name = "이 지 형 "
serverName = '127.0.0.1'
serverPort = 12000

clientSocket = socket(AF_INET, SOCK_DGRAM)

print("========================================")
print(f" UDP Client Running [{student_id} {student_name}]")
print(" (종 료 하 려 면  'q' 또는  'exit'를  입 력 하 세 요 )")
print("========================================₩n")
while True:
    message = input("Input lowercase sentence: ")
    if message.lower() in ['q', 'exit']:
        print("₩n[Client] 통 신 을  종 료 합 니 다 .")
        break
    full_message = f"[{student_id} {student_name}] {message}"
    clientSocket.sendto(full_message.encode(), (serverName, serverPort))
    modifiedMessage, serverAddress = clientSocket.recvfrom(2048)
    print(f"From Server: {modifiedMessage.decode()}₩n")

clientSocket.close()
```

1-3. UDP client-server프로그램 실행결과

```
User@DESKTOP-3CL4DB2 MINGW64 ~/OneDrive/바 탕  화 면 /2학 년 2학 기 /데 이 터  통 신 /UDP-TCP so
cket Programing (main)
$ py UDP_client.py
========================================
 UDP Client Running [2371214 이  지  형  ]
 (종 료 하 려 면  'q' 또 는  'exit'를  입 력 하 세 요 )
========================================

Input lowercase sentence: hello sever
From Server: [2371214 이  지  형  ] HELLO SEVER

Input lowercase sentence: week07 homework
From Server: [2371214 이  지  형  ] WEEK07 HOMEWORK

Input lowercase sentence: homework finish
From Server: [2371214 이  지  형  ] HOMEWORK FINISH

Input lowercase sentence: exit

[Client] 통 신 을  종 료 합 니 다 .

User@DESKTOP-3CL4DB2 MINGW64 ~/OneDrive/바 탕  화 면 /2학 년 2학 기 /데 이 터  통 신 /UDP-TCP so
cket Programing (main)
$
```

```
User@DESKTOP-3CL4DB2 MINGW64 ~/OneDrive/바 탕  화 면 /2학 년 2학 기 /데 이 터  통 신 /UDP-TCP so
cket Programing (main)
$ py UDP_server.py
========================================
 UDP Server Started by [2371214 이 지 형 ]
 The server is ready to receive (Port: 12000)
========================================

[Received from ('127.0.0.1', 64026)]: [2371214 이  지  형  ] hello sever
[Received from ('127.0.0.1', 64026)]: [2371214 이  지  형  ] week07 homework
[Received from ('127.0.0.1', 64026)]: [2371214 이  지  형  ] homework finish
```

1-4.UDP프로그램 설명

-UDP는 TCP와 달리 연결설정과정이 없는 비연결형 프로토콜로, 데이터 전송 신뢰성 보다는 신속한 패킷전달에 목적을 둡니다. 해당 프로그래밍 과제는 클라이언트가 입력한 소문자 문자열을 서버로 전송하면, 서버가 이를 대문자로 변환 하여 클라이언트에게 다시 반환하도록 설계 하였으며, 데이터통신 2장-7절의 PDF강의 자료를 참고하여 제작하였습니다.

-socket(AF_INET, SOCK_DGRAM): IPv4 주소 체계와 UDP프로토콜을 사용하는 소켓을 생성합니다.

-serverSocket.bind(('', serverPort))를 통해 서버가 클라이언트의 접속을 받아들일 포트 번호(12000)를 명시적으로 바인딩합니다.

-while True 무한 루프를 통해 서버 프로세스가 종료되지 않고 계속해서 요청을 받을 수 있도록 가동 상태를 유지하며. serverSocket.recvfrom(2048)을 사용하여 최대 2048 Byte 크기의 버퍼로부터 데이터그램 패킷을 수신합니다. 이때 수신받은 바이트 데이터와 상대방의 주소정보를 동시에 반환받습니다.

-수신 받은 데이터는 .decode()로 문자열반환후 출력하고, .upper()메서드로 소문자를 대문자로 반환한 메시지는 다시 .encode()과정을 거쳐서 바이트형태로 바꾼뒤, sendto()함수를 사용해 수신된 주소로 응답을 송신합니다.

2-1.TCP_server.py 소스코드 제출

```
from socket import *

student_id = "2371214"
student_name = "이 지 형 "
serverPort = 12000

# TCP 소켓 생성 (SOCK_STREAM 사용)
serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.bind(('', serverPort))
# 클라이언트의 연결 요청 대기
serverSocket.listen(1)
print("=========================================")
print(f" TCP Server Started by [{student_id} {student_name}]")
print(f" The server is ready to receive (Port: {serverPort})")
print("=========================================₩n")
```

```
while True:
    # 클라이언트의 연결 요청 수락 (전용 연결 소켓 생성)
    connectionSocket, addr = serverSocket.accept()
    sentence = connectionSocket.recv(1024).decode()
    print(f"[Received from {addr}]: {sentence}")
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.encode())
    connectionSocket.close()
```

2-2. TCP_client.py 소스코드 제출

```
from socket import *

student_id = "2371214"
student_name = "이 지 형 "
serverName = '127.0.0.1'
serverPort = 12000

print("=========================================")
print(f" TCP Client Running [{student_id} {student_name}]")
print(" (종 료 하 려 면  'q' 또는  'exit'를  입 력 하 세 요 )")
print("=========================================₩n")
```

```
while True:
    sentence = input("Input lowercase sentence: ")
    if sentence.lower() in ['q', 'exit']:
        print("₩n[Client] 통 신 을  종 료 합 니 다 .")
        break

    # TCP 소켓 생성 및 서버 연결
    clientSocket = socket(AF_INET, SOCK_STREAM)
    clientSocket.connect((serverName, serverPort))

    full_sentence = f"[{student_id} {student_name}] {sentence}"
    clientSocket.send(full_sentence.encode())

    # 서버 응답 수신
    modifiedSentence = clientSocket.recv(1024)
    print(f"From Server: {modifiedSentence.decode()}₩n")
    clientSocket.close()
```

2-3. TCP client-server프로그램 실행결과

```
User@DESKTOP-3CL4DB2 MINGW64 ~/OneDrive/바 탕  화 면 /2학 년 2학 기 /데 이 터  통 신 /UDP-TCP so
cket Programing (main)
$ py TCP_client.py
========================================
 TCP Client Running [2371214 이  지  형  ]
 (종  료  하  려  면    'q' 또 는    'exit'를   입  력  하  세  요  )
========================================

Input lowercase sentence: this is tcp homework
From Server: [2371214 이  지  형  ] THIS IS TCP HOMEWORK

Input lowercase sentence: hello server
From Server: [2371214 이  지  형  ] HELLO SERVER

Input lowercase sentence: homework finish
From Server: [2371214 이  지  형  ] HOMEWORK FINISH

Input lowercase sentence: exit

[Client] 통  신  을   종  료  합  니  다  .

User@DESKTOP-3CL4DB2 MINGW64 ~/OneDrive/바 탕  화 면 /2학 년 2학 기 /데 이 터  통 신 /UDP-TCP so
cket Programing (main)
$
```

```
User@DESKTOP-3CL4DB2 MINGW64 ~/OneDrive/바 탕  화 면 /2학 년 2학 기 /데 이 터  통 신 /UDP-TCP so
cket Programing (main)
$ py TCP_server.py
========================================
 TCP Server Started by [2371214 이  지  형  ]
 The server is ready to receive (Port: 12000)
========================================

[Received from ('127.0.0.1', 56646)]: [2371214 이  지  형  ] this is tcp homework
[Received from ('127.0.0.1', 56648)]: [2371214 이  지  형  ] hello server
[Received from ('127.0.0.1', 56694)]: [2371214 이  지  형  ] homework finish
```

2-4.TCP프로그램 설명

-TCP는 UDP와 달리 통신 전 3-Way Handshake 과정을 거쳐 가상 연결을 설정하는 연결 지향형(Connection-oriented) 프로토콜로서, 데이터 전송의 신뢰성, 순서 보장, 흐름 제어 및 혼잡 제어를 제공하는 특징을 가집니다.. 해당 TCP과제는 강의자료 2장-7절PDF 내용을 참고하여 클라이언트가 입력한 소문자 문자열을 서버로 전송하면, 서버가 이를 대문자로 변환하여 클라이언트에게 다시 반환하도록 설계하였습니다.

-socket(AF_INET, SOCK_STREAM)을 호출하여 IPv4 주소 체계와 TCP 스트림 프로토콜을 사용하는 소켓을 생성합니다.

-serverSocket.listen(1)을 실행하여 클라이언트의 TCP 연결 요청(SYN)을 받아들일 수 있는 대기 상태로 전환합니다.

-connectionSocket.recv(1024)를 통해 바이트 데이터를 수신하고 .decode()로 디코딩을 한 후에 터미널에 로그를 출력한다.

-.upper() 메서드로 대문자로 변환된 문자열을 .encode()로 인코딩과정을 거쳐 connectionSocket.send()로 클라이언트에 전송합니다.

-이번과제를 수행하면서 UDP와 다르게 사전 연결 과정(connect(), accept())이 반드시 필요하며, 데이터 송수신시 주소 정보를 매번 명시하지 않고 send()와 recv() 함수를 통해 신뢰성 있는 통신이 수행됨을 알수있었습니다.

-과제참고문헌(AI TOOL): GOOGLE GEMINI PLUS – 3.6FLASH